

Sistema de Gestión de Usuarios con Python, Tkinter y SQLite

Interfaz gráfica y base de datos local

01 · 09 · 2026



Del registro disperso a una gestión confiable

DOCUMENTOS + HOJAS

Consultar, modificar o eliminar información se vuelve lento, frágil y difícil de auditar.

APLICACIÓN LOCAL

Una solución tecnológica centraliza los datos y facilita operaciones rápidas y seguras.

Gestionar el ciclo completo del usuario

Una aplicación de escritorio que reúne formulario, operaciones CRUD y seguimiento visual.



Tecnologías simples. Resultado funcional.

PY

Python

Lógica de la aplicación

UI

Tkinter + ttk.Treeview

Formulario y tabla dinámica

DB

SQLite · sqlite3

Persistencia local

<>00

Visual Studio Code

Entorno de desarrollo

Archivos del proyecto: main.py + BaseUsuario.db

Una tabla central, consultas seguras

usuarios concentra identidad, contacto, estado, tipo, comentarios y archivos. El ID es clave primaria autoincremental; las operaciones usan parámetros.

id · nombre · apellido · teléfono · dirección · ciudad · código_postal · correo · género · estado · tipo_usuario · comentarios · imagen · archivo

CONFIGURACIÓN DE VENTANA

```

11  # =====
12  # CONFIGURACIÓN
13  # =====
14
15  raiz = Tk()
16  raiz.title("Sistema de Gestión de Usuarios")
17  raiz.geometry("1150x700")
18  raiz.resizable(False, False)
19
20  # Color azul claro
21  COLOR_FONDO = "#DCEEFF"
22
23  raiz.configure(bg=COLOR_FONDO)
24
25  try:
26      raiz.iconbitmap("Escudo_Junior.ico")
27  except:
28      pass

```

CONEXIÓN + CREATE TABLE

```

# =====
# CONEXIÓN A BASE DE DATOS
# =====

def conexion_bbdd():

    conexion = sqlite3.connect("DB_SanchoPan.db")
    cursor = conexion.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS usuarios (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nombre TEXT NOT NULL,
            apellido TEXT NOT NULL,
            telefono TEXT,
            direccion TEXT,
            ciudad TEXT,
            codigo_postal TEXT,
            correo TEXT,
            genero TEXT,
            estado INTEGER,
            tipo_usuario TEXT,
            comentarios TEXT,
            imagen TEXT,
            archivo TEXT
        )
    """)

    cursor.execute("PRAGMA table_info(usuarios)")

    columnas_bbdd = [
        columna[1]
        for columna in cursor.fetchall()
    ]

    if "comentarios" not in columnas_bbdd:

        cursor.execute(
            "ALTER TABLE usuarios ADD COLUMN comentarios TEXT"
        )

    conexion.commit()
    conexion.close()

```


Todo el flujo en una sola vista

Sistema de Gestión de Usuarios

Menú Ayuda Salir

FORMULARIO DE REGISTRO DE USUARIOS

Nombre: Angel

Dirección: Cra6A

Código Postal: 130003

Apellido: Pere

Ciudad: Malmabo

Género:

Masculino

Femenino

Teléfono: 3008609517

Correo: migulprz@gmail.com

Comentarios: El mejor

Usuario activo

Seleccionar Imagen

Adjuntar Archivo

Archivo: No adjunto

Tipo de usuario: Docente

JUNIOR

+ INSERTAR

ACTUALIZAR

SALIR

Buscar:

BUSCAR

MOSTRAR TODOS

ID	Nombre	Apellido	Teléfono	Dirección	Ciudad	Código Postal	Correo	Género	Estado	Tipo Usuario	Comentario
5	Angel	Pere	3008609517	Cra6A	Malmabo	130003	migulprz@gmail.com	Masculino	Activo	Docente	El mejor

Registro

Usuario registrado correctamente.

Aceptar

Validar primero. Insertar después.

Los campos críticos son obligatorios. Si todo está completo, un INSERT parametrizado registra al usuario y confirma el éxito.

VALIDACIONES

```
# =====
# INSERTAR
# =====
def insertar():
    if nombre.get() == "":
        messagebox.showwarning("Advertencia","Debe ingresar el nombre.")
        return
    if apellido.get() == "":
        messagebox.showwarning("Advertencia","Debe ingresar el apellido.")
        return
    if telefono.get() == "":
        messagebox.showwarning("Advertencia","Debe ingresar el teléfono.")
        return
    if direccion.get() == "":
        messagebox.showwarning("Advertencia","Debe ingresar la dirección.")
        return
    if ciudad.get() == "":
        messagebox.showwarning("Advertencia","Debe ingresar la ciudad.")
        return
    if codigo_postal.get() == "":
        messagebox.showwarning("Advertencia","Debe ingresar el código postal.")
        return
    if correo.get() == "":
        messagebox.showwarning("Advertencia","Debe ingresar el correo." )
        return
    if tipo_usuario.get() == "Seleccione":
        messagebox.showwarning("Advertencia","Debe seleccionar el tipo de usuario." )
        return
```

CONFIRMACIÓN EN LA INTERFAZ

Menú Ayuda Salir

FORMULARIO DE REGISTRO DE USUARIOS

Nombre: Angel

Dirección: Cra6A

Código Postal: 130003

Apellido: Pere

Ciudad: Malmabo

Género: ☒ Masculino
☐ Femenino

Teléfono: 3008609517

Correo: migulprz@gmail.com

Comentarios: El mejor

☒ Usuario activo

Tipo de usuario: Docente

Seleccionar Imagen

Adjuntar Archivo

Archivo: No adjunto

+ INSERTAR

ACTUALIZAR

SALIR

Buscar:

BUSCAR

MOSTRAR TODOS

ID	Nombre	Apellido	Teléfono	Dirección	Ciudad	Código Postal	Correo	Género	Estado	Tipo Usuario	Comentario
5	Angel	Pere	3008609517	Cra6A	Malmabo	130003	migulprz@gmail.com	Masculino	Activo	Docente	El mejor

Registro

Usuario registrado correctamente.

Aceptar

Buscar sin perder el contexto

FUNCIÓN BUSCAR

```
# =====
# BUSCAR
# =====

def buscar():
    texto = entrada_buscar.get()
    for elemento in tabla.get_children():
        tabla.delete(elemento)
    conexion = sqlite3.connect("DB_SanchoPan.db")
    cursor = conexion.cursor()
    cursor.execute("""
        SELECT
            id, nombre, apellido,
            telefono, direccion, ciudad,
            codigo_postal, correo, genero,
            estado, tipo_usuario, comentarios,
            imagen, archivo
        FROM usuarios
        WHERE nombre LIKE ?
            OR apellido LIKE ?
            OR ciudad LIKE ?
        ORDER BY id DESC
    """, (
        "%" + texto + "%", "%" + texto + "%", "%" + texto + "%"
    ))

    registros = cursor.fetchall()
    conexion.close()
    for registro in registros:
        estado_texto = ("Activo" if registro[9] == 1 else "Inactivo")
        tabla.insert(
            "",
            END,
            values=(
                registro[0],registro[1],registro[2],registro[3],registro[4],registro[5],
                registro[6],registro[7],registro[8],estado_texto,registro[10],registro[11])
        )
```

La consulta usa LIKE sobre nombre, apellido o ciudad.
Treeview se actualiza con coincidencias y avisa cuando no existen resultados.

RESULTADO FILTRADO

Buscar:	Angel	 BUSCAR		
ID	Nombre	Apellido	Teléfono	Dirección
5	Angel	Pere	3008609517	Cra6A

Editar con selección y control

El registro se selecciona desde la tabla, se carga en el formulario y se modifica mediante UPDATE parametrizado. Sin selección, la acción se detiene.

SQL UPDATE + VALIDACIÓN

```
1012 # =====
1013 # ACTUALIZAR
1014 # =====
1015
1016 def actualizar():
1017     if id_seleccionado.get() == "":
1018         messagebox.showwarning(
1019             "Advertencia", "Seleccione primero un registro.")
1020         return
1021     texto_comentarios = entrada_comentarios.get(
1022         "1.0",
1023         END
1024     ).strip()
1025     conexion = sqlite3.connect("DB_SanchoPan.db")
1026     cursor = conexion.cursor()
1027     cursor.execute("""
1028         UPDATE usuarios
1029         SET
1030             nombre = ?, apellido = ?, telefono = ?, direccion = ?,
1031             ciudad = ?, codigo_postal = ?, correo = ?, genero = ?,
1032             estado = ?, tipo_usuario = ?, comentarios = ?, imagen = ?,
1033             archivo = ?
1034         WHERE id = ?
1035     """, (
1036         nombre.get(), apellido.get(),
1037         telefono.get(), direccion.get(), ciudad.get(), codigo_postal.get(),
1038         correo.get(), genero.get(), estado.get(), tipo_usuario.get(),
1039         texto_comentarios, ruta_imagen.get(), ruta_archivo.get(), id_seleccionado.get()
1040     ))
1041     conexion.commit()
1042     conexion.close()
1043     messagebox.showinfo("Actualizar", "Registro actualizado correctamente.")
1044     mostrar_datos()
1045     limpiar()
1046
```

CONFIRMACIÓN

FORMULARIO DE REGISTRO DE USUARIOS

Nombre:

Apellido:

Teléfono:

Dirección:

Ciudad:

Correo:

Código Postal:

Género: ☒ Masculino ☐ Femenino

Comentarios:

☒ Usuario activo

Tipo de usuario:

Archivo: No adjunto

Buscar:

ID	Nombre	Apellido	Teléfono	Dirección	Ciudad	Código Postal	Correo
5	Angel Miguel	Perez Gomez	3008609517	Cra6A	Malmabo	130003	migulprz@gmail.com

Actualizar

Registro actualizado correctamente.

Aceptar

Una acción crítica, siempre confirmada

FUNCIÓN ELIMINAR

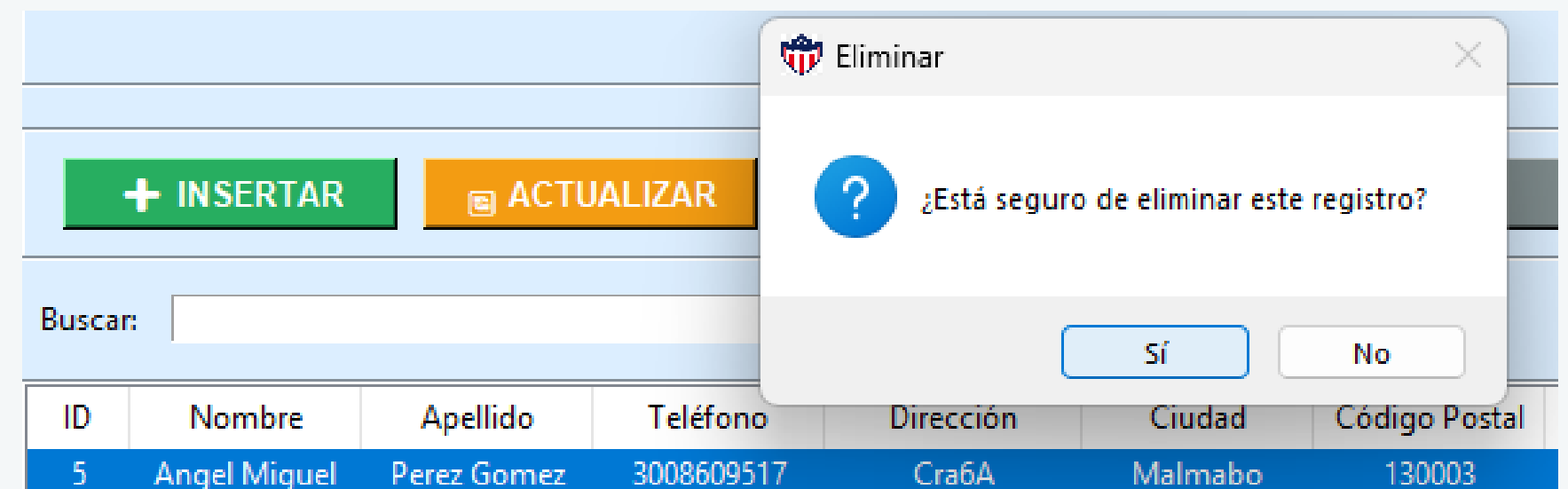
```

1049 # =====
1050 # ELIMINAR
1051 # =====
1052
1053 def eliminar():
1054
1055     if id_seleccionado.get() == "":
1056         messagebox.showwarning("Advertencia", "Seleccione un registro.")
1057         return
1058     respuesta = messagebox.askyesno(
1059         "Eliminar",
1060         "¿Está seguro de eliminar este registro?"
1061     )
1062     if respuesta:
1063         conexion = sqlite3.connect("DB_SanchoPan.db")
1064         cursor = conexion.cursor()
1065         cursor.execute(
1066             "DELETE FROM usuarios WHERE id = ?",
1067             (id_seleccionado.get(),)
1068         )
1069         conexion.commit()
1070         conexion.close()
1071         messagebox.showinfo("Eliminar", "Registro eliminado correctamente.")
1072         mostrar_datos()
1073         limpiar()

```

La aplicación exige selección previa, solicita confirmación y ejecuta DELETE parametrizado. Después, refresca la tabla para reflejar el cambio.

DIÁLOGO DE SEGURIDAD



Los errores se anticipan, no se esconden

```
if nombre.get() == "":
    messagebox.showwar
    return
if apellido.get() == "
    messagebox.showwar
    return
if telefono.get() == "
    messagebox.showwar
    return
if direccion.get() ==
    messagebox.showwar
    return
if ciudad.get() == "":
    messagebox.showwar
    return
if codigo_postal.get()
    messagebox.showwar
    return
if correo.get() == "":
    messagebox.showwar
    return
if tipo_usuario.get()
    messagebox.showwar
```

CAMPOS CRÍTICOS

Nombre, apellido, teléfono, dirección, ciudad, código postal, correo y tipo de usuario no pueden quedar vacíos.

BÚSQUEDA SEGURA

El ID se valida como numérico y los mensajes orientan la corrección antes de consultar la base.

Treeview convierte registros en seguimiento

MOSTRAR DATOS

```
# =====
# FRAME TREEVIEW
# =====

frame_tabla = Frame(raiz,bg=COLOR_FONDO,bd=2,relief="groove")
#groove significa un relieve hundido al borde del frame
frame_tabla.pack(padx=10,pady=5,fill="both",expand=True)

scroll_vertical = Scrollbar(frame_tabla,orient=VERTICAL)
scroll_vertical.pack(side=RIGHT,fill=Y)

scroll_horizontal = Scrollbar(frame_tabla,orient=HORIZONTAL)
scroll_horizontal.pack(side=BOTTOM,fill=X)
```

Todas las columnas relevantes permanecen visibles; los registros aparecen por ID descendente y cada fila puede cargar sus datos al formulario.

TABLA POBLADA

+ INSERTAR ACTUALIZAR ELIMINAR LIMPIAR SALIR							
Buscar: BUSCAR MOSTRAR TODOS							
ID	Nombre	Apellido	Teléfono	Dirección	Ciudad	Código Postal	Correo
5	Angel	Pere	3008609517	Cra6A	Malmabo	130003	migulprz@gmail.com
4	Elin	Ospino	3005647895	Calle 11	barranquilla	130003	Elinosp@gmail.com
3	Diego	Perez	30455584628	Calle 13	Puerto colombi	130003	Dieguitoprz@gmail.cor
2	Shadya	Cogollo	3011235647	calle 7	Salgar	130003	shandya@hotmail.com
1	Andru	Salas	3015632569	Calle 6	Puerro Colombi	1300003	AndxSalas@gmail.com

De funciones mínimas a una estructura escalable

HOY

Funciones CRUD + estado de interfaz

EVOLUCIÓN

Aplicación

Usuario

BaseDatos

MVC • SOLID

StringVar y estados conectan la interfaz actual. La siguiente versión separa responsabilidades con POO, SOLID y MVC.

Mejorar sin perder la simplicidad

01 Búsquedas avanzadas

Múltiples campos y filtros combinados

02 Visibilidad y control

Contador, contraseñas y confirmaciones

03 Experiencia profesional

Scroll, diseño y estados claros

04 Código mantenible

Responsabilidades bien separadas

GUI amigable + datos locales = aprendizaje aplicable

El proyecto demuestra un sistema funcional y didáctico para crear, consultar, actualizar y eliminar usuarios con una experiencia clara.

¿Preguntas o comentarios?

